REGISTERS & COUNTERS

B113040045 許育菖

2023/12/08

EXERCISE 1:

FOUR-BIT UNIVERSAL SHIFT REGISTER

題目說明

Mode Control		
s_1	s_0	Register Operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load

- Design and verify the four-bit universal shift register using Verilog HDL

  structural (gate-level) modeling

- Write an Verilog HDL stimulus to
  - (1) reset all flip-flops to 0
  - (2) parallel load “1111”,
  - (3) no change
  - (4) shift right => “0011”
  - (5) shift left => “1100”

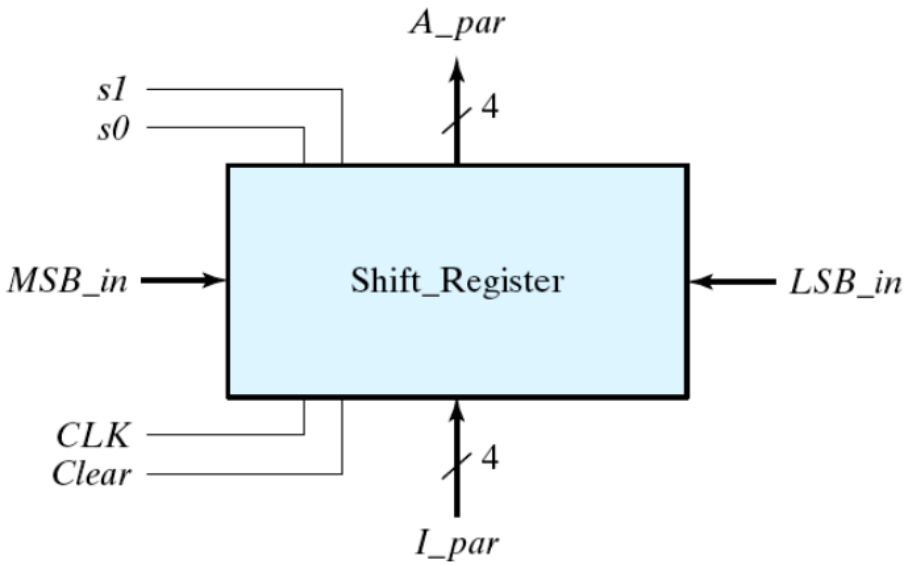


Fig. 6.7 Four-bit universal shift register

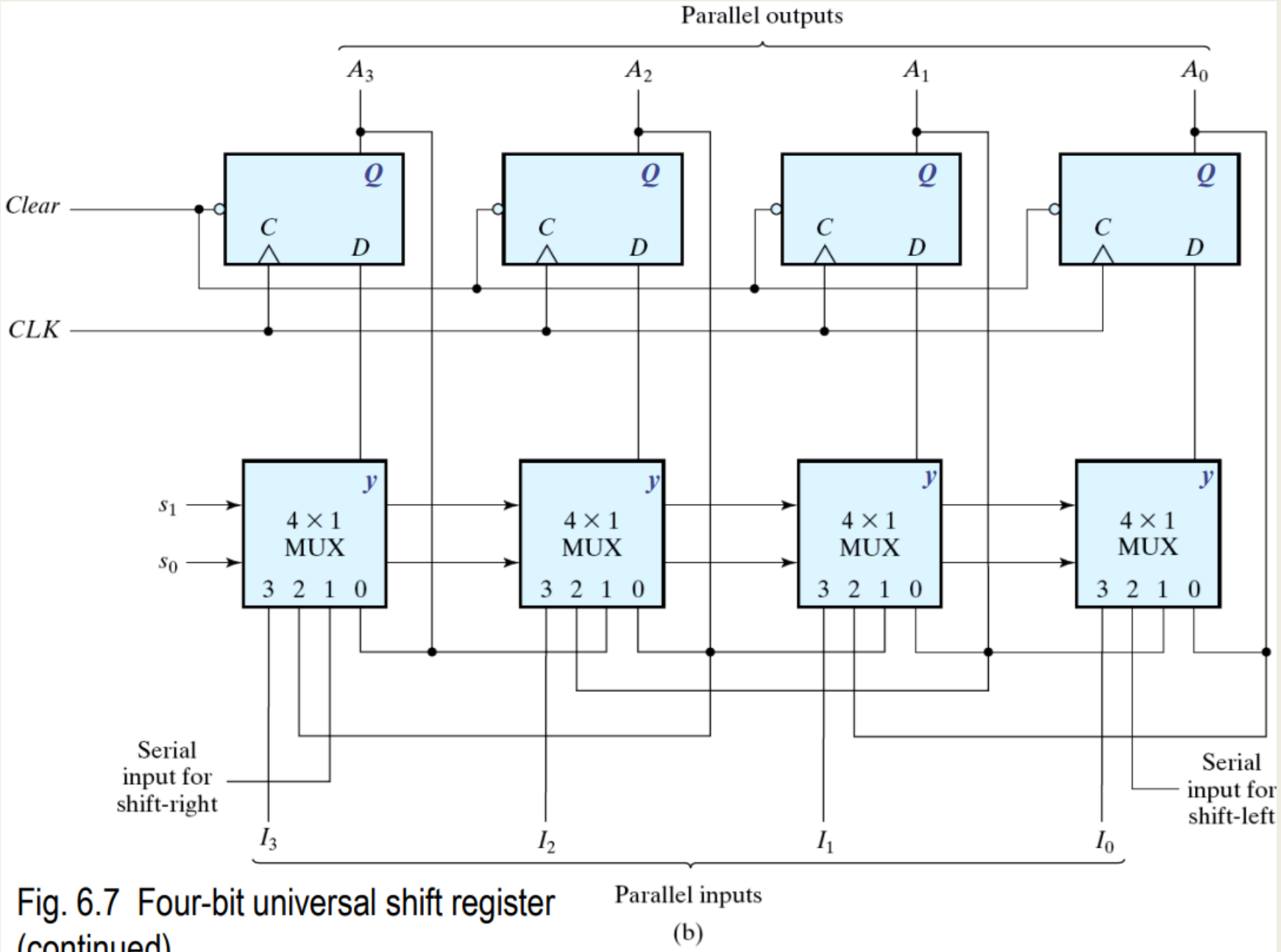


Fig. 6.7 Four-bit universal shift register (continued)

Verilog code

先分別寫好multiplexer和D flip-flop的structure level modeling，接著根據 **Fig 6.7** 的電路圖，將multiplexer和D flip-flop的輸出輸入進行連接。注意電路圖中multiplexer的輸入輸出看似十分混亂，但其中有一個「遞增」的規律存在

```
`timescale 1ns / 1ps
module exercise2(
    output [3:0] A,
    input [3:0] I,
    input S0, S1, right, left, CLK, Clear);

    wire [3:0] y;
    wire [3:0] Q;

    mux m0(y[0], Q[0], Q[1], left, I[0], S0, S1);
    mux m1(y[1], Q[1], Q[2], Q[0], I[1], S0, S1);
    mux m2(y[2], Q[2], Q[3], Q[1], I[2], S0, S1);
    mux m3(y[3], Q[3], right, Q[2], I[3], S0, S1);

    D_flip_flop D0(Q[0], y[0], CLK, Clear);
    D_flip_flop D1(Q[1], y[1], CLK, Clear);
    D_flip_flop D2(Q[2], y[2], CLK, Clear);
    D_flip_flop D3(Q[3], y[3], CLK, Clear);

endmodule
```

Verilog code

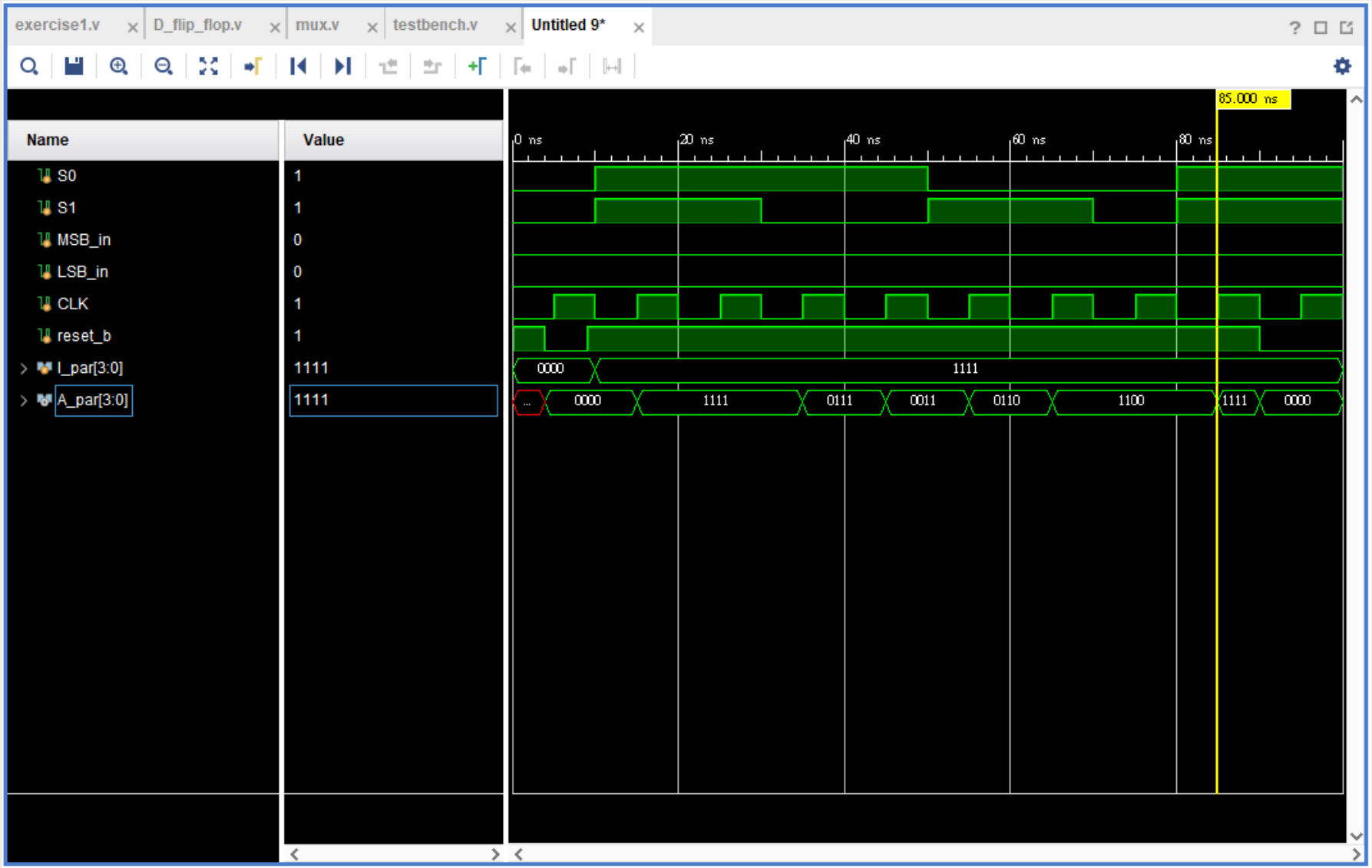
```
// D flip-flop
`timescale 1ns / 1ps

module D_flip_flop(Q, D, Clk, rst);
    output Q;
    input D, Clk, rst;
    reg Q;

always @ (posedge Clk, negedge rst)
if (rst == 0)
    Q <= 1'b0;
else
    Q <= D;

endmodule
```

```
// mux
`timescale 1ns / 1ps

module mux(
    output reg y,
    input I0, I1, I2, I3,
    input S0, S1);

    always @ (I0, I1, I2, I3, S0, S1)
        case (S0 + 2 * S1)
            2'b00: y = I0;
            2'b01: y = I1;
            2'b10: y = I2;
            2'b11: y = I3;
        endcase
endmodule
```

Testbench

```
`timescale 1ns / 1ps
module testbench ();
reg s1, s0, // Select inputs
MSB_in, LSB_in, // Serial inputs
clk, reset_b; // Clock and Clear_b
reg [3: 0] I_par; // Parallel input
wire [3: 0] A_par; // Register output
Exercise1 U0 (A_par, I_par,s0, s1, MSB_in, LSB_in, clk, reset_b);
initial #100 $finish;
initial begin clk = 0; forever #5 clk = ~clk; end
initial fork // test reset action load
   #3 reset_b = 1;
   #3 reset_b = 0;
   #9 reset_b = 1;
   // test parallel load
   #10 I_par = 4'b1111;
   #10 {s1, s0} = 2'b11;
   // test shift right
   #30 MSB_in = 1'b0;
   #30 {s1, s0} = 2  'b01;
   // test shift left
   #50 LSB_in = 1'b1;
   #50 {s1, s0} = 2'b10;
   // test circulation of data
   #70 {s1, s0} = 2'b00;
   // test reset on the fly
   #80 reset_b = 1'b1;
   #80 {s1, s0} = 2'b11;
   #90 reset_b = 1'b0;
join
endmodule
```

實驗結果


exercise1.v
D_flip_flop.v
mux.v
testbench.v
Untitled 9*
Name
Value
S0
1
S1
1
MSB_in
0
LSB_in
0
CLK
1
reset_b
1
I_par[3:0]
1111
A_par[3:0]
1111
85.000 ns
0 ns
20 ns
40 ns
60 ns
80 ns
0000
1111
0000
1111
0111
0011
0110
1100
1111
0000

EXERCISE 2:

BCD SYNCHRONOUS COUNTER

題目說明

- Design and verify the BCD synchronous counter using **T flip flops** with asynchronous reset and Verilog HDL **structural (gate-level) modeling**
- Write an Verilog HDL stimulus to

  (1) reset all flip-flops to 0

  (2) count from “0000” to “0000”

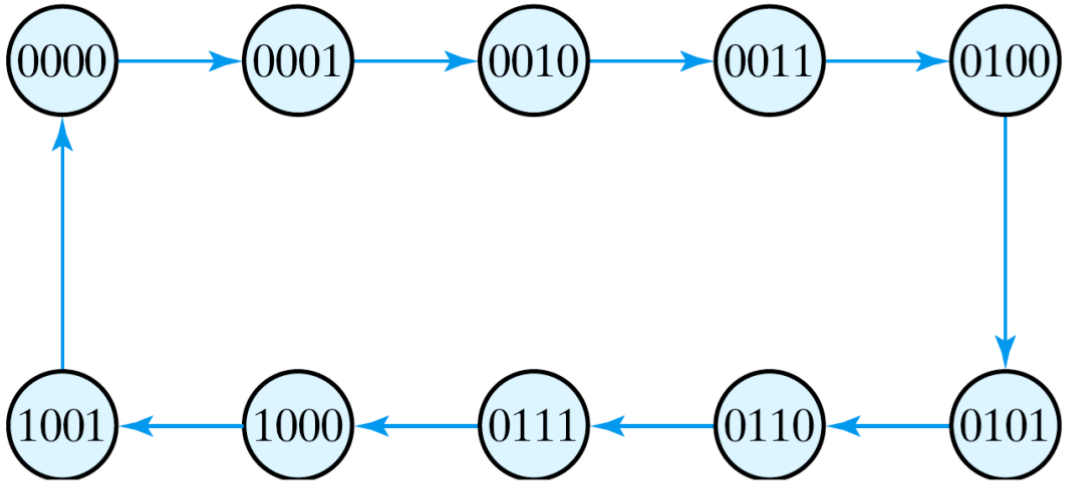


Fig. 6.9 State diagram of a decimal BCD counter

Table 6.5
State Table for BCD Counter

Present State				Next State				Output	Flip-Flop Inputs			
Q_8	Q_4	Q_2	Q_1	Q_8	Q_4	Q_2	Q_1	y	TQ_8	TQ_4	TQ_2	TQ_1
0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	1	0	0	1	0	0	0	0	1	1
0	0	1	0	0	0	1	1	0	0	0	0	1
0	0	1	1	0	1	0	0	0	0	1	1	1
0	1	0	0	0	1	0	1	0	0	0	0	1
0	1	0	1	0	1	1	0	0	0	0	1	1
0	1	1	0	0	1	1	1	0	0	0	0	1
0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	0	0	1	0	0	0	0	1
1	0	0	1	0	0	0	0	1	1	0	0	1

經過計算可以得到：

$$TQ_8 = Q_8Q_1 + Q_4Q_2Q_1$$
$$TQ_4 = Q_2Q_1$$
$$TQ_2 = Q_4Q_1$$
$$TQ_1 = 1$$

Verilog code

從 **Table 6.5** 的真值表可以分別計算出TQ_8、TQ_4、TQ_2、TQ_1的公式。再將這四個變數分別帶入T flip-flop，即可next state的值

```
// T flip-flop
`timescale 1ns / 1ps
module T_flipflop(Q, T, Clk, rst);
    output Q;
    input T, Clk, rst;
    reg Q;

always @ (posedge Clk, negedge rst)
if (rst == 0)
    Q <= 1'b0;
else
    Q <= T ^ Q;
endmodule
```

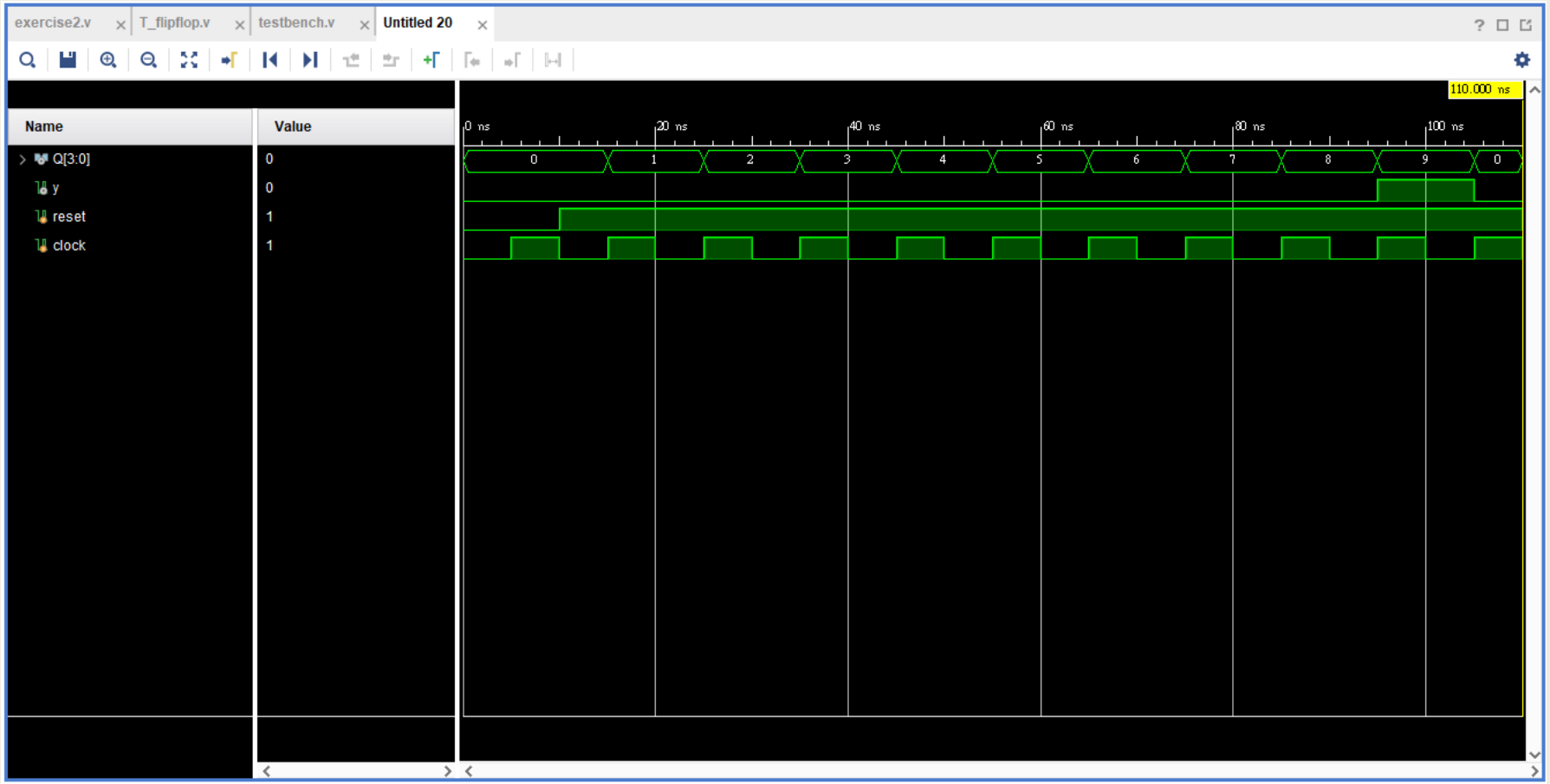
```
`timescale 1ns / 1ps
module exercise2(
    output [3:0] A,
    output y,
    input reset, clock );

    wire TQ8, TQ4, TQ2, TQ1;

    assign TQ8 = (A[3] & A[0]) | (A[2] & A[1] & A[0]);
    assign TQ4 = A[1] & A[0];
    assign TQ2 = (!A[3]) & A[0];
    assign TQ1 = 2'b1;

    T_flipflop T8(A[3], TQ8, clock, reset);
    T_flipflop T4(A[2], TQ4, clock, reset);
    T_flipflop T2(A[1], TQ2, clock, reset);
    T_flipflop T1(A[0], TQ1, clock, reset);

    assign y = A[3] & A[0];
endmodule
```

Testbench

因為是BCD counter，所以此題的testbench只要處理reset和clock的值即可

```
`timescale 1ns / 1ps

module testbench();
    wire [3:0] Q;
    wire y;
    reg reset, clock;

exercise2 U2(Q, y, reset, clock);

initial #110 $finish;
initial begin clock = 0; forever #5 clock = ~clock; end

initial begin
    reset = 1'b1;
    reset = 1'b0;
    #10 reset = 1'b1;
end
endmodule
```

實驗結果


exercise2.v
T_flipflop.v
testbench.v
Untitled 20
Name
Value
Q[3:0]
y
reset
clock
0
0
1
1
110.000 ns
0 ns
20 ns
40 ns
60 ns
80 ns
100 ns
0
1
2
3
4
5
6
7
8
9
0

實驗心得

這次實驗我花費滿多時間的，雖然第一題是以正常速度寫完，但第二題光搞懂題目意思就花了滿多時間，甚至不知道如何下手寫testbench。

我覺得第二題會這麼困難的主要原因是，以前的題目都是從testbench控制輸入與輸出，然而第二題的BCD counter僅能直接從testbench控制clock和reset的值，類似於間接控制整個電路輸入與輸出，型式上與之前的題目有些不同，因而容易讓人搞混。

不過，最讓我心態爆炸的是，程式撰寫完後波形圖竟然跑不出來！而且還找不出電路哪裡有邏輯錯誤。最後才發現是有個部份我把&打成%了 ==，真的超無言。